Kubernetes in Action

Chapter 2: Docker와 Kubernetes 첫 걸음

Kubernetes in Action 1st Edition — Marko Lukša

현재 K8s 버전(1.35+) 기준 업데이트 포함

목차

1. **Docker 기초** — 이미지, 컨테이너, 레이어
2. **BusyBox** — 초경량 유틸리티
3. **Dockerfile & 이미지 빌드** — 레이어 최적화
4. **컨테이너 격리** — Namespace & PID
5. **Kubernetes 클러스터 구성** — Minikube, GKE
6. **첫 번째 앱 배포** — Deployment, Service, Scaling
7. **동작 원리 심화** — ReplicaSet, Service, Endpoints
8. **1판 vs 현재** — Breaking Changes

1. Docker 기초

Docker Hello World

```
docker run busybox echo "Hello world"
```

```
docker run busybox echo "Hello world"
     │
     ├─ 1) 로컬에 busybox 이미지가 있는지 확인
     ├─ 2) 없으면 Docker Hub에서 pull
     ├─ 3) 이미지로부터 컨테이너 생성
     ├─ 4) 컨테이너 안에서 echo "Hello world" 실행
     └─ 5) 프로세스 종료 → 컨테이너 중지
```

- **이미지**: 앱 + 환경을 레이어로 패키징한 읽기 전용 템플릿
- **컨테이너**: 이미지의 실행 인스턴스 (이미지 + 쓰기 가능 레이어)

2. BusyBox

BusyBox — The Swiss Army Knife

> 수백 개의 Unix 유틸리티를 **단일 바이너리 (~1.2MB)** 에 담은 초경량 도구

```
일반 Linux:                          BusyBox:
  /bin/ls     (120KB)                  /bin/busybox (~1.2MB)
  /bin/cp     (140KB)                    ← 300+ 명령어 내장
  /bin/cat    (50KB)
  /bin/grep   (200KB)
  ... 합계 수십 MB
```

동작 원리: `argv[0]` (호출된 이름)을 보고 해당 명령 실행

```
ln -s /bin/busybox /bin/ls     # 심볼릭 링크
ls -la                          # busybox가 ls 모드로 동작
busybox ls -la                  # 직접 호출도 가능
```

컨테이너 베이스 이미지 비교

이미지	크기	셸	패키지 매니저	용도
`busybox`	~1.2MB	ash	-	디버깅, init container
`alpine`	~5MB	ash	apk	범용 경량 베이스
`distroless`	~2-20MB	-	-	프로덕션 보안
`scratch`	0MB	-	-	Go/Rust 정적 바이너리

K8s 디버깅 활용:

```
kubectl run debug --image=busybox -it --rm -- sh

# Pod 안에서:
wget -qO- http://my-service:8080/health   # HTTP 요청
nslookup my-service                        # DNS 확인
nc -zv my-db 5432                          # 포트 확인
```

3. Dockerfile & 이미지 빌드

이미지 레이어 구조

Docker 이미지 = **읽기 전용 레이어들의 스택**

```
| Writable Layer          |   ← 컨테이너 실행 시 추가 (R/W)
| ENTRYPOINT node app     |   Layer 4 (Read-Only)
| RUN npm install         |   Layer 3 (Read-Only)
| COPY . /app             |   Layer 2 (Read-Only)
| FROM node:20-alpine     |   Layer 1 (Base)
```

Dockerfile의 각 명령어(`FROM`, `RUN`, `COPY`)가 **하나의 레이어**를 생성

나쁜 예 vs 좋은 예

나쁜 예

```
FROM node:20-alpine
WORKDIR /app
COPY . .
RUN npm install
ENTRYPOINT ["node", "app.js"]
```

코드 1줄 변경 → `npm install` **매번 재실행**

좋은 예

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --production
COPY . .
ENTRYPOINT ["node", "app.js"]
```

코드 변경 시 `npm ci` **캐시 히트!**

레이어 캐시 흐름

소스코드만 수정했을 때:

Layer	명령	결과
1	`FROM node:20-alpine`	✅ 캐시 히트
2	`COPY package*.json`	✅ 캐시 히트
3	`RUN npm ci`	✅ 캐시 히트
4	`COPY . .`	🔄 재빌드

> 빌드 시간 **수 분 → 수 초**

레이어 최적화 원칙

원칙	이유
변경 빈도 낮은 것 → **위에**	캐시 히트율 극대화
변경 빈도 높은 것 → **아래에**	캐시 무효화 범위 최소화
RUN 명령 **합치기**	불필요한 레이어 수 줄이기
.dockerignore 활용	COPY 레이어 크기 줄이기

RUN 합치기

```
# 나쁜 예 — 레이어 3개 (중간 레이어에 캐시 찌꺼기 남음)
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# 좋은 예 — 레이어 1개
RUN apt-get update && apt-get install -y curl \
    && rm -rf /var/lib/apt/lists/*
```

Multi-stage Build

빌드 도구는 최종 이미지에 **불필요** — 스테이지를 분리

```dockerfile
# ── Stage 1: Build ──
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ── Stage 2: Runtime (빌드 결과물만 복사) ──
FROM node:20-alpine
WORKDIR /app
RUN npm ci --production
COPY --from=builder /app/dist ./dist
ENTRYPOINT ["node", "dist/main.js"]
```

	크기	포함 내용
Single-stage	800MB	node_modules + 빌드도구

4. 컨테이너 격리

PID Namespace

컨테이너 안의 프로세스는 **자신만의 PID 공간**에서 실행된다

```
호스트 OS 관점:                    컨테이너 관점:
  PID 1    : systemd                 PID 1 : node app.js
  PID 3150 : node app.js             ← "나는 PID 1이다"
```

```
docker exec -it kubia-container bash
ps aux
# PID 1 = node app.js (컨테이너의 메인 프로세스)
```

- 컨테이너 안: **자기 프로세스만** 보임
- 호스트에서: 컨테이너 프로세스도 보임 (다른 PID)
- 격리 메커니즘: **Linux Namespace** (PID, Network, Mount, IPC, UTS, User)

5. Kubernetes 클러스터 구성

Minikube — 로컬 클러스터

```
minikube start                    # 단일 노드
minikube start --nodes 3          # 멀티 노드 (v1.10+)
minikube start --cni=calico       # CNI 플러그인 지정
```

```
┌──── 로컬 머신 ──────────────────────────┐
│                                          │
│   ┌──── Minikube VM/Container ────┐      │
│   │  Control Plane + Worker       │      │
│   │  (단일 노드에 전부)            │      │
│   └───────────────────────────────┘      │
│                                          │
│  kubectl ◄──── ~/.kube/config            │
└──────────────────────────────────────────┘
```

kubectl 연결

`minikube start` → **kubeconfig 자동 설정** → kubectl이 API Server에 mTLS 연결

GKE — 클라우드 클러스터

```
gcloud container clusters create kubia --num-nodes 3

kubectl get nodes
# NAME                          STATUS   ROLES    AGE
# gke-kubia-default-pool-xxx-1  Ready    <none>   1m
# gke-kubia-default-pool-xxx-2  Ready    <none>   1m
# gke-kubia-default-pool-xxx-3  Ready    <none>   1m
```

kubeconfig 구조

항목	역할
cluster	API 서버 주소 + CA 인증서
user	클라이언트 인증서/키
context	cluster + user 조합

```
kubectl config get-contexts          # 모든 context 확인
kubectl config use-context gke-prod  # context 전환
```

6. 첫 번째 앱 배포

앱 배포 — Deployment 생성

⚠️ **1판 변경**: `kubectl run --generator` 는 **삭제됨** (K8s 1.21+)

```
# 1판 (에러 발생)
# kubectl run kubia --image=luksa/kubia --port=8080 --generator=run/v1

# 현재 방식
kubectl create deployment kubia --image=luksa/kubia --port=8080
```

```
kubectl create deployment kubia
    │
    ├─ 1) Deployment 'kubia' 생성
    ├─ 2) Deployment → ReplicaSet 생성
    ├─ 3) ReplicaSet → Pod 생성
    ├─ 4) Scheduler → 워커 노드에 배정
    └─ 5) kubelet → containerd로 컨테이너 실행
```

Service로 외부 노출

```
kubectl expose deployment kubia --type=LoadBalancer --name kubia-http
```

```
인터넷 → LoadBalancer (104.155.74.57:8080)
            |
            ▼
        Service (kubia-http)
            |
            ▼
        Pod (kubia-xxxxx) → Container (luksa/kubia)
```

이유	설명
Pod는 **일시적**	죽으면 새 IP로 재생성됨
Service = **고정 엔드포인트**	IP가 변하지 않음
로드밸런싱	뒤의 Pod들로 트래픽 분산

수평 스케일링

```
kubectl scale deployment kubia --replicas=3
```

```
Service (kubia-http) ─── LoadBalancer
      │
      ├──► Pod kubia-xxxxx (Node 1)
      ├──► Pod kubia-yyyyy (Node 2)
      └──► Pod kubia-zzzzz (Node 1)
```

```
curl 104.155.74.57:8080   # You've hit kubia-xxxxx
curl 104.155.74.57:8080   # You've hit kubia-zzzzz
curl 104.155.74.57:8080   # You've hit kubia-yyyyy
```

> 앱 코드 변경 없이, **명령어 한 줄**로 스케일링
> K8s Scheduler가 노드 리소스를 보고 **자동 배치**

시스템의 논리적 구성

```
┌─ Deployment ──────────────────────────────────────────┐
│  name: kubia                                          │
│                                                       │
│   ┌─ ReplicaSet ──────────────────────────────────┐   │
│   │  replicas: 3                                  │   │
│   │                                               │   │
│   │   ┌─ Pod ─┐   ┌─ Pod ─┐   ┌─ Pod ─┐          │   │
│   │   │ kubia │   │ kubia │   │ kubia │          │   │
│   │   └───────┘   └───────┘   └───────┘          │   │
│   └───────────────────────────────────────────────┘   │
└───────────────────────────────────────────────────────┘
                  │ label: app=kubia
                  ▼
┌─ Service (kubia-http) ──┐
│  type: LoadBalancer     │
│  ClusterIP: 10.96.0.100 │
│  External: 104.155.74.57│
└─────────────────────────┘
```

7. 동작 원리 심화

ReplicaSet — Control Loop

"이 label을 가진 Pod가 항상 N개 실행되도록 보장"

```
ReplicaSet 제어 루프:

  1) label=app:kubia인 Pod 수 확인
        │
        ├─ 부족 → Pod 생성
        ├─ 초과 → Pod 삭제
        └─ 딱 맞음 → 대기
        │
  2) watch로 실시간 감시, 반복
```

자가 치유 시나리오

```
정상:     Pod-1 ── Pod-2 ── Pod-3     RS: "3/3 OK"
장애:     Pod-1 ── Pod-2 ── Pod-3 ✗   RS: "2/3! Pod-4 생성"
복구:     Pod-1 ── Pod-2 ── Pod-4     RS: "3/3 OK"
```

Deployment → ReplicaSet → Pod

Deployment가 **모든 것을 관리**하는 현재 표준 구조:

```
Deployment (선언적 업데이트, 롤백)
    │
    └─► ReplicaSet (Pod 수 유지, 자동 생성됨)
            │
            ├─► Pod-1
            ├─► Pod-2
            └─► Pod-3
```

오브젝트	직접 생성?	역할
Deployment	✅ 직접	롤링 업데이트, 롤백, RS 관리
ReplicaSet	❌ 자동	Pod 수를 replicas 만큼 유지
Pod	❌ 자동	실제 컨테이너 실행 단위

> RS를 직접 만들 일은 없다. 1판의 RC는 레거시.

Deployment — 롤링 업데이트

```
kubectl set image deployment/kubia kubia=luksa/kubia:v2
```

```
시점 1:  RS-v1(3)  RS-v2(0)   ← 업데이트 시작
시점 2:  RS-v1(2)  RS-v2(1)   ← v2 Pod 1개 올라옴
시점 3:  RS-v1(1)  RS-v2(2)   ← 순차 교체 중
시점 4:  RS-v1(0)  RS-v2(3)   ← 완료

→ Zero Downtime 버전 교체
```

롤백: `kubectl rollout undo deployment/kubia`

→ 기존 RS(replicas=0으로 보존)를 다시 scale up

Service 동작 원리

`kubectl expose`가 생성하는 것

```yaml
apiVersion: v1
kind: Service
metadata:
  name: kubia-http
spec:
  type: LoadBalancer
  selector:
    app: kubia            # ← Pod label과 매칭
  ports:
    - port: 8080
      targetPort: 8080
```

> Service는 Deployment/RS와 **직접 연결되지 않는다**
> **label selector**로 Pod를 찾는다

Service 타입별 외부 노출

```
ClusterIP (기본)   — 클러스터 내부에서만 접근
   Pod → ClusterIP (10.96.x.x) → Pod

NodePort           — 모든 노드의 특정 포트
   Client → <NodeIP>:31348 → Service → Pod

LoadBalancer       — 클라우드 LB + NodePort + ClusterIP
   Client → LB (외부IP) → NodePort → Service → Pod
```

타입	접근 범위	사용처
`ClusterIP`	클러스터 내부	Pod 간 통신 (기본값)
`NodePort`	노드IP:포트	개발/테스트
`LoadBalancer`	외부 IP	프로덕션 (클라우드)
`ExternalName`	DNS CNAME	외부 서비스 연결

트래픽 흐름 상세

```
curl 104.155.74.57:8080
     │
     ▼
LoadBalancer
     │
     ▼
NodePort :31348
     │
     ▼
kube-proxy (iptables/IPVS)
     │
     │   Endpoints:
     │     - 10.244.0.5:8080  (Pod-1)
     │     - 10.244.1.3:8080  (Pod-2)
     │     - 10.244.0.6:8080  (Pod-3)
     │
     ▼
랜덤 Pod 선택 → "You've hit kubia-yyyyy"
```

전체 동작 흐름

```
┌─ API Server ──────────────────────────────────────────┐
│                                                        │
│  Deployment Controller → ReplicaSet 생성               │
│  ReplicaSet Controller → Pod 3개 생성                  │
│  Scheduler → 각 Pod를 노드에 배정                      │
│  Endpoints Controller → Pod IP 수집                    │
└────────────────────────────────────────────────────────┘
                          │
                ┌─────────┴─────────┐
                ▼                   ▼
   ┌─ Node 1 ──────────┐   ┌─ Node 2 ──────────┐
   │ kubelet           │   │ kubelet           │
   │  → Pod-1, Pod-3   │   │  → Pod-2          │
   │ kube-proxy        │   │ kube-proxy        │
   │  → iptables       │   │  → iptables       │
   └───────────────────┘   └───────────────────┘
                │                   │
                └── LoadBalancer ───┘
                        │
                  외부 클라이언트
```

8. 1판 vs 현재 — Breaking Changes

Breaking Changes

항목	1판 (K8s ~1.9)	현재 (1.35+)
`kubectl run`	Deployment 생성	**Pod만 생성**
`--generator`	핵심 기능	**삭제 (에러)**
`--replicas` in run	작동	**삭제 (에러)**
`docker ps` on node	작동	**불가** (dockershim 제거)
Dashboard	`kubectl apply`	**프로젝트 아카이브**

Changed / Deprecated

항목	1판	현재
RC → Deployment	주요 예제가 RC	Deployment + ReplicaSet
Docker 런타임	유일한 선택	containerd / CRI-O
minikube 드라이버	VirtualBox	Docker / vfkit
Dashboard	기본 관리 도구	Headlamp 대체

현재 방식 명령어 정리

```
# ❌ 1판 (에러)
kubectl run kubia --image=luksa/kubia --generator=run/v1
kubectl scale rc kubia --replicas=3          # ← RC 기반 (레거시)
kubectl expose rc kubia --type=LoadBalancer  # ← RC 기반 (레거시)

# ✅ 현재
kubectl create deployment kubia --image=luksa/kubia --port=8080
kubectl scale deployment kubia --replicas=3
kubectl expose deployment kubia --type=LoadBalancer --name kubia-http
```

dockershim 제거 (K8s 1.24+)

```
1판:    kubelet → dockershim → Docker Engine → 컨테이너
현재:   kubelet → CRI → containerd → 컨테이너
```

> Docker로 **빌드한 이미지**는 여전히 어디서든 동작 (OCI 표준)

Docker 대안: Podman

특징	Docker	Podman
데몬	dockerd 필요	Daemonless
권한	기본 root	Rootless
CLI	`docker ...`	`podman ...` (호환)
Pod 지원	Compose	네이티브 Pod

```
alias docker=podman     # 거의 그대로 사용 가능
```

요약

Chapter 2 핵심 정리

1. **Docker 이미지** = 읽기 전용 레이어 스택, 변경 빈도순 배치가 핵심
2. **BusyBox** = 300+ 명령어를 1.2MB에 담은 초경량 도구
3. **Multi-stage Build**로 이미지 크기를 80%+ 절감
4. **컨테이너 격리** = Linux Namespace + cgroups (Docker가 아닌 커널 기능)
5. **Deployment → ReplicaSet → Pod** 3계층 구조가 현재 표준
6. **Service** = label selector로 Pod를 찾아 고정 엔드포인트 + 로드밸런싱
7. **kubectl expose** = Service 자동 생성, Endpoints가 Pod IP 관리
8. 1판의 `--generator`, RC(ReplicationController) 기반 명령어는 **모두 Deployment 방식으로 변경됨**

Q&A

Thank you!